

[L0-L1 Core Essence & Design Logic]

L0 Essence: An serverless, zero-configuration relational database engine that compiles declarative SQL into VDBE register bytecode, abstracts I/O through B-Trees and Pager caching, and ensures ACID transaction isolation via OS file-locking interfaces.

- L1 Logic:**
- Parsing & Code Generation** (M1) utilizes a specialized Lemon generator to generate reentrant, stack-efficient parsers, flattening the AST and emitting optimized VDBE bytecode.
 - VDBE Register Machine** (M2) executes a sequential stream of pseudo-instructions via a giant switch-case loop, manipulating database cursors directly to avoid Volcano-style overhead.
 - B-Tree & Pager Layer** (M3) coordinates adaptive overflow allocation and PCache lifecycle (Clean/Dirty/Pinned), converting block I/O into abstract key-value lookups.
 - Rollback Journal & WAL** (M4 & M5) manage transaction rollback and shared-memory indexing, allowing progress from exclusive write locks to concurrent, non-blocking reads/writes.

[L2 SQLite Kernel Data Flow Topology]

• [SQL Statement] → [Lemon AST Parser] → [Prepared VDBE Bytecode] → [VDBE Interpretive Loop] → [VFS Lock Acquisition] → [PCache Page Mapping] → [B-Tree Page Cell Seek] → [WAL Frame Append] → [Checkpoint Sync to DB]

[M1.1] Tokenizer & Lemon Parser Generator

- **Reentrant Parser Generator:** SQLite rejects Bison/Yacc in favor of the custom Lemon parser generator. Lemon constructs LALR(1) parsers that are reentrant and lack global state variables.
- **Low Stack Footprint:** Lemon parameterizes grammar actions, minimizing stack size during parsing to protect low-resource embedded devices from stack overflows.

[M1.2] Code Generator & Query Flattening

- **Instruction Generation:** The code generator performs AST traversals to output flat VDBE register bytecode.
- **Query Flattening:** Optimizations flatten nested SELECT statements and subqueries into single-level joins, eliminating intermediate temporary table allocations during VM execution.

[M1.3] Cost-Based Optimizer & Index Selection

- **Heuristic Cost Estimation:** The query planner estimates cost boundaries by calculating expected physical page scans.
- **Expression Indexes:** Supports indexes built directly on computed expressions, replacing AST expression trees during optimization to avoid runtime computation.

[M1.4] Prepared Statement Cache

- **Compilation Reuse:** The `sqlite3_prepare_v2` call compiles SQL strings into pre-allocated VDBE programs with unbound parameters (e.g., `?`).
- **Zero-Allocation Binding:** Developers execute queries using `sqlite3_bind_*` and `sqlite3_step` loops, bypassing parsing and optimizer phases.

[M2.1] VDBE Architecture & Opcode Loop

- **Register-Based Soft CPU:** VDBE is a register-based virtual machine that treats operations as sequences of pseudo-machine instructions.
- **Central Dispatch Loop:** The execution engine in `sqlite3VdbeExec()` runs as a giant switch-case loop that dispatches opcodes via program counter offsets.

[M2.2] VDBE Registers & Dynamic Typing

- **Dynamic Mem Cells:** VDBE registers are represented by `Mem` struct cells. They support dynamic typing (NULL, INTEGER, REAL, TEXT, BLOB) and handle pointer reuse.
- **Type Affinity:** Registers automatically cast data types at runtime based on column affinity rules, allowing flexible schema designs.

[M2.3] Cursor Operations & B-Tree Interface

- **Cursor Handles:** VDBE uses cursors to read and write records in B-Tree pages.
- **B-Tree Seeking:** Opcodes like `OpenRead` allocate cursors, `SeekRowId` executes binary searches, and `Column` extracts values directly from B-Tree cells.

[M2.4] Bytecode Subroutines & Coroutines

- **Context Switching:** VDBE avoids deep call stacks by using `Yield` and `Gosub` instructions to act as light coroutines.
- **Temporary Table Avoidance:** In sorting and GROUP BY operations, VDBE yields rows dynamically to consumers, minimizing memory buffers.

[M2.5] VDBE Explain Command & Analysis

- **Query Plan Diagnostics:** Executing `EXPLAIN QUERY PLAN <SQL>` outputs query execution plans, documenting index use (e.g., `USING INDEX`).
- **Bytecode Disassembly:** Using `EXPLAIN <SQL>` prints the complete disassembly of VDBE opcodes (e.g., `Init`, `OpenRead`, `Column`, `ResultRow`).

[M3.1] B-Tree Page Structure & Layout

- **Node Alignments:** B-Tree page sizes range from 512B to 64KB. Headers contain 8 to 12 bytes of metadata indicating node status (leaf or interior).
- **Cell Organization:** Cells (keys and payload data) grow from the page end toward the top, while offsets are stored in cell pointer arrays growing from the header downward.

[M3.2] Overflow Pages

- **Large Payload Eviction:** If a cell's payload exceeds page-specific thresholds, SQLite evicts the excess data to overflow pages.
- **Link Pointer Chains:** The primary leaf cell retains only a portion of the payload and a 4-byte pointer to the overflow chain, preserving node fan-out.

[M3.3] Pager Logic & PCache Management

- **Page to Memory Mapping:** The Pager sits between VDBE/B-Tree and OS. It maps file blocks into memory Page nodes tracked by the PCache.
- **Pinning & Dirty Flags:** Pager locks pages in memory by pinning them, and flags modified buffers as dirty to trigger writes.

[M3.4] Free-list & Compaction

- **Page Deallocation:** When data is deleted, page offsets are appended to a global free-list rather than truncating the database file.
- **Physical Compaction:** The `VACUUM` command creates a clean database copy on disk and copies active pages, removing fragments and shrinking file size.

[壹] SQLite Concurrency & Journaling Trade-offs & Fallacies Matrix

Sharing a single database handle across multiple processes maximizes throughput: Multi-process sharing of a single `sqlite3` pointer disrupts OS lock states, corrupting database pages or leading to process crashes. → Open a separate database connection handle for each process. In multi-threaded environments, configure a connection pool or use thread-local handles.

Deleting the `-journal` or `-wal` file is a safe way to resolve locked database errors: Deleting active transaction logs removes Page metadata, leaving the database in a partially written, unrecoverable state. → Never touch journal or WAL files manually. Instead, identify and terminate the hung process holding the OS lock, letting SQLite auto-recover.

Setting `synchronous = OFF` is a harmless way to increase write speed: Setting `synchronous` to `OFF` bypasses OS file system `fsync` barriers, making the database vulnerable to corruption during power losses. → Configure WAL mode with `synchronous = NORMAL`. This restricts `fsync` overhead to WAL checkpoints, balancing transaction safety and performance.

[M4.1] -journal File Layout

- **Original State Cache:** In rollback mode, Pager writes original page contents to a `~journal1` file before any page modifications occur.
- **Journal Header:** The header specifies magic numbers, transaction page counts, and the original database file size for rollbacks.

[M4.2] Rollback Mode Commit & Recovery

- **Two-Phase Write Barrier:**
 1. **Log Sync:** Write original pages to the journal and issue `fsync` on the journal.
 2. **DB Update:** Write dirty pages to the database file and issue `fsync` on it. Delete/truncate the journal file.
- **Automatic Rollback:** If a crash occurs mid-write, Pager reads the `~journal1` file on restart to restore original pages to the database.

[M4.3] Lock States in Rollback Mode

- **Lock Hierarchy:**
 1. **SHARED:** Readers read the file. No writes allowed.
 2. **RESERVED:** One writer plans to write. Readers can read.
 3. **PENDING:** Writer waits for active readers to finish. Blocks new readers.
 4. **EXCLUSIVE:** Writer holds exclusive access, allowing page writes to disk.

[M4.4] Multi-process Write Constraints

- **Concurrency Block:** Rollback writes require an EXCLUSIVE file lock, blocking all readers and returning SQLITE_BUSY.

[M5.1] -wal File Layout

- **Append-Only Log:** WAL mode writes updates to a separate `-wal` file, keeping the main database file untouched during transactions.
- **WAL Frame Headers:** The file starts with a 32-byte header, followed by frames containing 24-byte headers and physical page payloads.

[M5.2] -shm Shared-Memory Index

- **Index Acceleration:** To avoid scanning gigabytes of WAL frames, SQLite generates a `-shm` index file mapping pages to WAL offsets.
- **Process Shared Map:** Different processes map the `-shm` file using `mmap` to locate page frames within microseconds.

[M5.3] WAL Read-Write Concurrency

- **Snapshot Isolation:** Writers append frames to WAL. Readers locate frames via the `-shm` index, reading from WAL or database.
- **Non-Blocking Concurrency:** Readers and a single writer run simultaneously without conflicts or locks.

[M5.4] Checkpoint Process

- **Dirty Page Sync:** Checkpointing reads frames from WAL and writes them back to the `.db` file in order, updating file states.
- **WAL Reset:** Resets the WAL write pointer. If readers are referencing old frames, a PASSIVE checkpoint leaves those frames in WAL until readers exit.

[M5.5] WAL fsync Rules

- **Fsync Barriers:**
 1. **Commit:** Flush dirty pages to WAL and execute `fsync` on the WAL file.
 2. **Checkpoint:** Write frames to DB, execute `fsync` on the DB file, write checkpoint flags to WAL, and sync the database.

[M5.6] WAL Page Eviction & Wraparound

- **Log Wraparound:** SQLite monitors the minimum read frame index across all active connections using `-shm` state tables.
- **Zero-Overhead Reuse:** When readers advance past the WAL header, writes reset to offset 0, preventing file growth.

[M6.1] VFS (Virtual File System) Abstraction

- **OS Agnostic Layer:** The VFS abstracts platform-specific storage APIs.
- **Interface Hooks:** The `sqlite3_vfs` structure exposes functions like `xOpen`, `xDelete`, `xRead`, `xWrite`, `xLock`, and `xUnlock` to keep kernel logic clean.

[M6.2] Windows vs Unix File Locking

- **Unix Byte Range Locking:** Unix VFS uses `fcntl` advisory locks on database byte ranges (starting at offset 1,073,741,824) to represent SHARED/PENDING/EXCLUSIVE locks.
- **Win32 LockFileEx:** Windows VFS uses `LockFileEx` to lock matching byte ranges, ensuring cross-process security.

[M6.3] Thread vs Process Locks

- **Mutex Protections:** Mutexes isolate thread modifications to database connections.
- **File Lock Barriers:** Process concurrency is managed by file lock tables, preventing concurrent modifications by different programs.

[M6.4] mmap I/O Optimization

- **Zero-Copy Access:** Configuring `PRAGMA mmap_size` maps database pages directly into the process's virtual memory address space.
- **Bypassing Kernels:** Eliminates read system calls and kernel-to-user memory copy operations, accelerating page reads.

[M6.5] SEE & SQLCipher Encryption

- **Page-Level Encryption:** SEE and SQLCipher encrypt pages (excluding the first 100 bytes of the database file) using AES-256 before writes.
- **Decryption Pipeline:** Pages are decrypted upon loading into PCache, presenting clear text to VDBE while maintaining encrypted files on disk.

[貳] Zone T: SQLite Diagnostic & Tuning Reference Laboratory		
T1 SQLite Typical Tuning Parameters		T2 Database Maintenance Commands • 1. Query Plan & Bytecode Diagnostics <pre>sql -- Disassembles and displays VDBE instructions to analyze VM registers and cursor walks EXPLAIN SELECT name, age FROM users WHERE age > 18; -- Displays high-level execution plans to analyze index hits and join topologies EXPLAIN QUERY PLAN SELECT * FROM orders JOIN users ON orders.uid = users.id;</pre> • 2. Database Integrity Verification <pre>sql -- Scans B-Tree nodes, cell layouts, and free-lists to identify physical page corruption PRAGMA integrity_check;</pre>